

Lecture 8: Intermediate Code Intro & IR

中文学习笔记

基于课件 7.Intermediate_Code_Intro_IR.pdf 整理

2026 年 6 月 26 日

目录

阅读导引	3
1 回顾与编译阶段定位 [p1–5]	3
1.1 从 SDD/SDT 到中间代码 [p2]	3
1.2 中间代码生成在编译器中的位置 [p3–5]	3
2 多层 IR 与使用多种 IR 的原因 [p6–9]	4
2.1 IR 的作用 [p6]	4
2.2 高层、低层和机器层 IR [p6–7]	4
2.3 为什么要多层 IR [p8–9]	5
3 中间表示与三地址码 [p10–18]	5
3.1 三类重要 IR [p10–11]	5
3.2 三地址码的基本形式 [p12]	6
3.3 优化动机：公共子表达式消除 [p13]	6
3.4 三地址码中的地址 [p14]	7
3.5 三地址指令形式 [p15–17]	7
3.6 do-while 示例 [p18]	7
4 三地址码的实现：四元式、三元式、间接三元式 [p19–30]	8
4.1 四元式 [p19–21]	8
4.2 三元式 [p22–24]	9
4.3 三元式的问题 [p25–26]	9

目录	2
4.4 间接三元式 [p27–30]	10
5 SSA：静态单赋值 [p31–33]	10
5.1 SSA 为什么利于优化	11
6 语法制导代码生成与变量布局 [p34–43]	11
6.1 语法制导翻译重新登场 [p34–36]	11
6.2 变量定义的属性 [p37]	12
6.3 作用域内分配与 offset [p38–39]	12
6.4 内存对齐 [p40]	13
7 赋值语句翻译 [p44–46]	13
7.1 基本属性和辅助函数	13
7.2 SDT 规则与例子	14
8 数组引用翻译 [p47–52]	14
8.1 一维数组 [p48]	14
8.2 多维数组 [p49–50]	15
8.3 数组引用属性和生成规则 [p51–52]	15
9 布尔表达式翻译 [p53–58]	16
9.1 短路求值 [p54]	16
9.2 两种翻译方式 [p55–57]	16
9.3 布尔表达式 SDT 规则 [p58]	17
10 控制流翻译与回填 [p59–70]	17
10.1 控制语句标签属性 [p61–63]	17
10.2 前向跳转问题 [p64–66]	18
10.3 回填属性和辅助函数 [p67]	18
10.4 控制流回填规则 [p68–70]	19
11 总结与阅读 [p71–73]	19
12 练习题与解答	20
复习建议	23

阅读导引

本讲进入编译器前端和后端之间的桥梁：中间代码生成（intermediate code generation）和中间表示（intermediate representation, IR）。前面的语法分析和语义分析已经确认程序结构合法、标识符绑定正确、类型信息可用；本讲开始把带标注的 AST 翻译成更适合优化和目标代码生成的 IR，重点是三地址码、SSA、语法制导代码生成、数组与控制流翻译，以及单遍生成中的回填。

本讲主线

IR 的价值在于把“源语言相关的结构”转换为“优化和后端更容易处理的结构”。三地址码把复杂表达式拆成简单指令，SSA 把值流显式化，回填解决单遍生成时前向跳转目标未知的问题。

1 回顾与编译阶段定位 [p1-5]

1.1 从 SDD/SDT 到中间代码 [p2]

课件开头回顾上一讲语义分析：

- SDD：属性、综合属性、继承属性、依赖图、拓扑序。
- S-SDD 和 L-SDD：分别适合 LR 和 LL/RDP 等不同实现方式。
- SDT：把语义动作放在产生式不同位置，执行翻译或检查。
- 应用：符号表、作用域、类型检查。

这些内容为本讲服务：中间代码生成也可以看成一种语法制导翻译。语义分析给 AST 加上类型、符号表入口、偏移等标注；代码生成规则根据这些属性输出三地址码。

1.2 中间代码生成在编译器中的位置 [p3-5]

编译阶段：

Source Code → Lexical Analyzer → Syntax Analyzer → Semantic Analyzer
→ Intermediate Code Generation → Code Optimizer → Code Generator.

中间代码生成的输入是 syntax tree 或 annotated/decorated AST, 输出是 intermediate representation。此时程序仍保持机器无关性，但已经不像 AST 那样高度依赖源语言表面语法。

课件用 while 代码说明翻译结果：

```
while (y < z) {  
    int x = a + b;
```

```
y += x;
}
```

可能翻译成类似三地址码：

```
goto L1
L2:
t1 := a + b
x := t1
t2 := y + x
y := t2
L1:
if y < z goto L2
```

这说明控制结构被转为标签和跳转，复杂表达式被转为临时变量上的简单运算。

2 多层 IR 与使用多种 IR 的原因 [p6-9]

2.1 IR 的作用 [p6]

Intermediate Representation

中间表示是编译过程中使用的程序表示形式，位于源代码和目标机器码之间。它既能保留足够语义信息，又比源语言语法更规整，便于优化和后端处理。

IR 的优势：

- 提高抽象层次，使每个编译阶段只处理适合自己的表示。
- 让前端、优化器、后端之间边界更清晰。
- 支持一个编译器构造一系列 IR，而不是只用一种表示贯穿始终。

2.2 高层、低层和机器层 IR [p6-7]

层级	例子	特点	主要用途
High-level IR	Parse Tree, AST	接近源语言，通常语言相关	语义分析、保留源程序结构

Low-level IR	Three-address code, SSA	接近抽象机器指令集，语言和机器相对独立	机器无关优化
Machine-level IR	x86 IR, ARM IR	接近具体 ISA，机器相关	寄存器分配、指令选择、目标代码生成

高层 IR 适合表达“这个程序是什么结构”；低层 IR 适合表达“这个程序如何一步步计算”；机器层 IR 适合表达“目标机器上怎么执行”。

2.3 为什么要多层 IR [p8-9]

只用一种 IR 是可以的。例如某些 JIT 编译器为了减少编译时间，可能在语义分析后直接从 AST 生成机器码。但多层 IR 有两个重要优势：

- 1. 更适合具体任务。语义分析用 AST 更自然，优化用 TAC/SSA 更自然，寄存器分配必须接近机器层。
- 2. 更容易扩展前端和后端。低层 IR 可作为多源语言和多目标机器之间的桥梁。

如果只有 AST，新增一个源语言就要为新 AST 重写所有优化；新增一个 ISA 也要写新的 AST 到机器码转换。若统一降到低层 IR，优化器可复用，后端也只需处理一种较统一的输入。

3 中间表示与三地址码 [p10-18]

3.1 三类重要 IR [p10-11]

课件列出几类重要 IR：

- 树形结构：parse tree、AST。
- DAG：有向无环图，可把共享子表达式表达为共享节点。
- 线性表示：尤其是三地址码。
- SSA：静态单赋值形式，常建立在低层 IR 上。

同一个程序片段可以同时表示为树和线性代码。例如：

```
do i = i + 1; while (a[i] < v);
```

树更直观表达循环、下标、比较等结构；三地址码则把它拆成编号指令、临时变量和跳转，便于优化器顺序扫描和重排。

3.2 三地址码的基本形式 [p12]

Three-Address Code

三地址码是一种线性 IR，每条指令右侧最多只有一个操作符，通用形式为：

$$X = Y \text{ op } Z.$$

其中 X, Y, Z 可以是变量名、常量或编译器生成的临时变量。

例子：

$$x + y * z$$

不能直接作为一条三地址指令，因为右侧有两个运算。应拆为：

```
t1 = y * z
t2 = x + t1
```

三地址码的特点：

- 是语法树或 DAG 的线性化表示。
- 可看成某种抽象机器的汇编语言。
- 长表达式被拆成多条简单指令。
- 控制流被转为条件跳转和无条件跳转。
- 与具体机器无关，适合机器无关优化。

3.3 优化动机：公共子表达式消除 [p13]

表达式：

$$a * b + a * b$$

直接翻译可得：

```
t1 = a * b
t2 = a * b
t3 = t1 + t2
```

优化器可识别重复的 $a * b$ ，做公共子表达式消除（common subexpression elimination, CSE）：

```
t1 = a * b
t3 = t1 + t1
```

三地址码比 AST 更容易暴露这类优化机会，因为每一步运算都以简单指令形式呈现。

3.4 三地址码中的地址 [p14]

三地址码中的地址可以是：

- 名字：源程序中的变量名、函数名等。
- 常量：整数、浮点数、字符串等。
- 编译器生成的临时变量：例如 t_1, t_2 。每次需要保存中间值时生成不同临时变量，便于优化和寄存器分配。

临时变量在最终代码生成时可能合并到寄存器或栈位置中。中间阶段保留独立临时名，可以让数据依赖更清楚。

3.5 三地址指令形式 [p15-17]

类型	形式	说明
二元赋值	$x = y \text{ op } z$	op是二元算术或逻辑运算符
一元赋值	$x = \text{op } y$	如 unary minus、逻辑非、shift、类型转换
复制	$x = y$	把 y 的值赋给 x
无条件跳转	goto L	下一条执行标签 L 处指令
条件跳转	if x goto L , ifFalse x goto L , if x relop y goto L	根据条件决定控制流
过程调用	param x , call p, n	先压参数，再调用函数 p , 参数数目为 n
返回	return y	返回值可选
下标复制	$x = y[i], x[i] = y$	从数组偏移处读写
地址与指针	$x = \&y, x = *y, *x = y$	取地址、解引用读、解引用写

课件中数组和指针形式强调：三地址码不仅能表达算术，还能表达内存地址计算。

3.6 do-while 示例 [p18]

源程序：

```
do {
    i = i + 1;
} while (a[i] < v)
```

对应 TAC：

```

L: t1 = i + 1
i = t1
t2 = &a
t3 = sizeof(int)
t4 = t3 * i
t5 = t2 + t4
t6 = *t5
if t6 < v goto L

```

这里 $a[i]$ 被翻译为：先取数组基地址，再用元素宽度乘以下标得到偏移，最后解引用得到元素值。

4 三地址码的实现：四元式、三元式、间接三元式 [p19-30]

4.1 四元式 [p19-21]

Quadruple

四元式用四个字段表示一条三地址指令：

$$(\text{op}, \text{arg1}, \text{arg2}, \text{result}).$$

例子：

$$x = y + z \Rightarrow (+, y, z, x).$$

复制语句 $x = y$ 中，op 为=。很多运算的赋值符号是隐含的，例如 $x = y + z$ 写作 $+, y, z, x$ 。

特殊情况：

- 一元操作 $x = \text{minus } y$: $(\text{minus}, y, _, x)$ ，不使用 arg2。
- param x : $(\text{param}, x, _, _)$ ，不使用 arg2 和 result。
- goto L : $(\text{goto}, _, _, L)$ ，目标标签放入 result。
- 条件跳转和无条件跳转都把目标标签放入 result。

例子：

$a = b * (-c) + b * (-c)$

可翻译为：

编号	op	arg1	arg2	result
0	minus	c		t1
1	*	b	t1	t2
2	minus	c		t3
3	*	b	t3	t4
4	+	t2	t4	t5
5	=	t5		a

4.2 三元式 [p22-24]

Triple

三元式只有三个字段：

$(op, arg1, arg2).$

它没有显式 result 字段，某条指令的结果隐含为该指令编号。

例如 $x = y + z$ 写成 $(+, y, z)$ ，结果由该三元式编号引用。上一例对应三元式：

编号	op	arg1	arg2
0	minus	c	
1	*	b	(0)
2	minus	c	
3	*	b	(2)
4	+	(1)	(3)
5	=	a	(4)

复杂左值需要额外三元式。例如 $x[i] = y$ ：

编号	op	arg1	arg2
0	[]	x	i
1	=	(0)	y

第 0 条先计算 $x[i]$ 的地址，第 1 条再把 y 写入该地址。

4.3 三元式的问题 [p25-26]

三元式节省了 result 字段，但优化时不方便。因为结果由指令位置表示，一旦移动或删除指令，所有引用该编号的位置都可能需要更新。

例如做 CSE 后删除重复计算，如果原来某条指令引用的是(4)，而第 4 条已经被删除或移动，就会出现悬空引用。优化器经常需要移动代码，因此纯三元式不利于代码变换。

4.4 间接三元式 [p27-30]

Indirect Triple

间接三元式把三元式存入一个 triple database，真正表示程序执行顺序的是一个指向三元式的指针列表。移动指令时只重排指针列表，不移动三元式本体。

优点：

- 指令重排更容易，不会破坏基于三元式编号的引用。
- CSE 删除指令后，数据库中的空洞可复用。
- 程序代码本身是“指令列表”，不是数据库中连续排列的三元式。

例子：

```
x = (a+b)*c;  
y = d/(a+b);
```

公共子表达式 $a + b$ 可只保留一个三元式，后续两处通过指针引用它。优化器若移动指令，只需改变 instruction list 中指针顺序。

5 SSA：静态单赋值 [p31-33]

Single Static Assignment

SSA 要求每个变量在静态程序文本中只被赋值一次。每次赋值都会生成一个新版本名，例如 x_1, x_2, x_3 。控制流汇合处用 ϕ 函数合并来自不同路径的版本。

例子：

```
x = a + b;  
y = x - c;  
x = x - y;  
if (...) x = x + 5;
```

```

else      x = x * 4;
y = x * 4;

```

可写成：

```

x1 = a + b
y1 = x1 - c
x2 = x1 - y1
if (...)
    x3 = x2 + 5
else
    x4 = x2 * 4
x5 = phi(x3, x4)
y2 = x5 * 4

```

$\phi(x_3, x_4)$ 表示：若控制流来自 true 分支，取 x_3 ；若来自 false 分支，取 x_4 。

5.1 SSA 为什么利于优化

SSA 能显式表达值流：

- 它告诉编译器什么时候不该优化。例如 $x_2 * 4$ 和 $x_5 * 4$ 看起来形式相似，但 SSA 版本不同，值可能不同，不能错误地做 CSE。
- 它告诉编译器什么时候应该优化。例如：

```

x1 = a + b
x2 = c - d
y1 = x2 * b

```

x_1 从未被使用，是明显死值，可被 DCE 删除。

- 没有 SSA 时，优化器通常需要额外数据流图来判断值的定义和使用；SSA 把这些信息直接写进 IR。

6 语法制导代码生成与变量布局 [p34-43]

6.1 语法制导翻译重新登场 [p34-36]

代码生成依赖程序语法结构，因此可以继续用 SDD/SDT 描述。要翻译的语言结构包括：

- 变量、函数等定义。
- 赋值语句。

- 数组引用。
- 布尔表达式。
- 控制流语句，例如 if、if-else、while、for。

课件采用的策略：

1. 写出 SDD 语义规则，说明要生成什么代码和属性。
2. 转成 SDT 动作，说明动作执行顺序。

函数代码生成分两步：

- 处理变量定义，决定变量在内存中的布局。
- 处理语句，为赋值、数组、布尔表达式和控制流生成 TAC。

6.2 变量定义的属性 [p37]

变量布局需要两个信息：

- location：变量位于内存哪里。
- width：变量占多少字节。

对产生式风格的声明 $T \ V$ ，可使用属性：

属性	含义
$T.type$	类型名，如 int、float
$T.width$	类型宽度，如 int 为 4 字节
$V.type$	变量类型
$V.width$	变量宽度，由类型决定
$V.offset$	变量在所属内存区域内的偏移

6.3 作用域内分配与 offset [p38-39]

最朴素的方法是为所有数据预留一个大内存区，变量地址等于该区基地址加变量 offset。但这会浪费空间，因为很多变量只在短时间内存活。例如函数局部变量只在函数调用期间存在。

更合理的方式是按作用域分配：

- 进入作用域时分配该作用域的存储区域。
- 离开作用域时释放该区域。
- 同一作用域内变量一起分配和释放。

- 变量地址 = 作用域基地址 + 变量 offset。

函数局部变量顺序布局示例：

```
void foo() {  
    int a;          // offset 0  
    int b;          // offset 4  
    long long c;    // offset 8  
    int d;          // offset 16  
}
```

若 int 为 4 字节，long long 为 8 字节，则最终 offset 到 20。

6.4 内存对齐 [p40]

Allocation Alignment

内存对齐要求变量地址满足：

$$\text{addr}(x) \bmod \text{sizeof}(x.\text{type}) = 0.$$

也就是说，变量起始地址最好落在其类型大小的整数倍边界上。

原因是多数机器在类型大小边界上访问效率最高。例如 int 若不在 word boundary 上，机器可能需要读两个 word，再移位拼接，效率较低。

例子：

```
struct { char x; int y; } Test;
```

若不对齐，大小可理解为 $1 + 4 = 5$ 。实际对齐时，char x 后会填充 3 字节，使 int y 从地址 4 开始，结构体总大小通常为 8。

7 赋值语句翻译 [p44–46]

7.1 基本属性和辅助函数

翻译赋值语句时，表达式会被拆成每条最多一个操作符的 TAC。常用辅助函数：

- lookup(id)：查符号表，返回标识符地址或条目；找不到则报错。
- emit()/gen()：生成三地址码。
- newtemp()：生成新的临时变量。

属性:

- $S.code$ 、 $E.code$: 语句或表达式对应的 TAC 序列。
- $E.addr$: 表达式值所在地址, 可能是变量名、常量或临时变量。

7.2 SDT 规则与例子

文法:

$$S \rightarrow id = E; \quad E \rightarrow E_1 + E_2 \mid -E_1 \mid (E_1) \mid id.$$

关键规则:

- $S \rightarrow id = E$: 先查 id , 再输出 $id = E.addr$ 。
- $E \rightarrow E_1 + E_2$: 生成新临时变量 t , 输出 $t = E_1.addr + E_2.addr$ 。
- $E \rightarrow -E_1$: 生成 $t = minus\ E_1.addr$ 。
- $E \rightarrow (E_1)$: 直接继承 $E_1.addr$ 和 $E_1.code$ 。
- $E \rightarrow id$: 查符号表得到地址, 代码为空。

对:

```
a = b + (-c)
```

生成:

```
t1 = minus c
t2 = b + t1
a = t2
```

增量翻译中, $gen()$ 不仅返回一条指令, 还把它追加到当前已生成指令序列中, 因此不必显式维护 $E.code$ 的字符串拼接。

8 数组引用翻译 [p47-52]

8.1 一维数组 [p48]

数组引用的核心是计算元素地址。一维数组:

```
int A[N];
A[i]++;
```

若 $base$ 是首元素地址, $width$ 是元素宽度, 则:

$$addr(A[i]) = base + i \times width.$$

8.2 多维数组 [p49-50]

C 语言使用 row-major layout, 即按行连续存储。二维数组:

$int\ A[N_1][N_2]$

的地址为:

$$addr(A[i_1, i_2]) = base + (i_1 \times N_2 \times width + i_2 \times width).$$

k 维数组一般形式:

$$addr(A[i_1][i_2] \cdots [i_k]) = base + i_1 W_1 + i_2 W_2 + \cdots + i_k W_k.$$

例子:

- $array(10, int)$: $a[i]$ 偏移 $i * 4$ 。
- $array(3, array(5, int))$: $a[i_1][i_2]$ 偏移 $i_1 * 20 + i_2 * 4$ 。
- $array(3, array(5, array(8, int)))$: $a[i_1][i_2][i_3]$ 偏移 $i_1 * 160 + i_2 * 32 + i_3 * 4$ 。

8.3 数组引用属性和生成规则 [p51-52]

对左值 L :

- $L.array$: 指向数组名对应的符号表条目, 能得到 base 地址。
- $L.type$: 当前子数组类型。
- $L.addr$: 当前已经计算出的偏移临时变量。

文法:

$$\begin{aligned} S &\rightarrow id = E \mid L = E, & E &\rightarrow E_1 + E_2 \mid -E_1 \mid (E_1) \mid id \mid L, \\ L &\rightarrow id[E] \mid L_1[E]. \end{aligned}$$

若 L 作为右值表达式, 生成:

$$E.addr = newtemp(); \quad gen(E.addr = L.array.base[L.addr]).$$

若 L 作为赋值左值, 生成:

$$gen(L.array.base[L.addr] = E.addr).$$

三维例子 $c = a[i_1][i_2][i_3]$:

```
t1 = i1 * 160
t2 = i2 * 32
t3 = t1 + t2
t4 = i3 * 4
t5 = t3 + t4
c = a[t5]
```

9 布尔表达式翻译 [p53-58]

9.1 短路求值 [p54]

布尔表达式包含关系运算和逻辑运算，例如 <、<=、!=、>、>=、&&、||、==。

Short-circuit Evaluation

短路求值指：一旦布尔表达式结果已经确定，就跳过后续不必要的求值。

例子：

- `if (flag || foo()) { bar(); }` 中，若 `flag` 为真，`foo()` 不执行。
- `if (flag && foo()) { bar(); }` 中，若 `flag` 为假，`foo()` 不执行。

因此在控制流中，布尔运算最好直接翻译成跳转，而不是一定先计算出 0/1 临时值。

9.2 两种翻译方式 [p55-57]

方式一：像算术表达式一样先算布尔临时变量。

对：

$$(a < b) \ || \ (c < d \ \&\& \ e < f)$$

可生成：

```
t1 = a < b
t2 = c < d
t3 = e < f
t4 = t2 && t3
t5 = t1 || t4
if (!t5) goto S.next
S1.code
S.next:
```

方式二：直接生成跳转，更适合短路控制流。

```
if a < b goto E.true
goto L1
L1: if c < d goto L2
goto E.false
```



```
L2: if e < f goto E.true
goto E.false
```

这种方式用两个标签属性：

- $E.true$ ：表达式为真时跳转的标签。
- $E.false$ ：表达式为假时跳转的标签。

9.3 布尔表达式 SDT 规则 [p58]

布尔规则：

- $B \rightarrow B_1 || B_2$ ：若 B_1 真，则整体真；若 B_1 假，才进入 B_2 。因此 $B_1.true = B.true$ ， $B_1.false$ 是 B_2 的开始标签。
- $B \rightarrow B_1 \wedge B_2$ ：若 B_1 假，则整体假；若 B_1 真，才进入 B_2 。因此 $B_1.false = B.false$ ， $B_1.true$ 是 B_2 的开始标签。
- $B \rightarrow E_1 \text{ relop } E_2$ ：直接生成条件跳转和失败跳转。
- $B \rightarrow !B_1$ ：交换 $B_1.true$ 和 $B_1.false$ 。
- $B \rightarrow true$ ：生成 `goto B.true`。
- $B \rightarrow false$ ：生成 `goto B.false`。

10 控制流翻译与回填 [p59-70]

10.1 控制语句标签属性 [p61-63]

控制语句包括：

$$S \rightarrow \text{if}(B)S_1, \quad S \rightarrow \text{if}(B)S_1 \text{ else } S_2, \quad S \rightarrow \text{while}(B)S_1.$$

继承属性：

- $B.true$ ：若 B 为真，控制流跳到哪里。
- $B.false$ ：若 B 为假，控制流跳到哪里。
- $S.next$ ：语句 S 之后紧跟的指令标签。

辅助函数：

- `newlabel()`：创建新标签。
- `label(L)`：把标签 L 附着到下一条 TAC 指令。

if-else 的结构:

```
B.code
B.true:
S1.code
goto S.next
B.false:
S2.code
S.next:
```

if 无 else 时, $B.false = S.next$ 。while 的结构:

```
S.begin:
B.code
B.true:
S1.code
goto S.begin
S.next:
```

10.2 前向跳转问题 [p64-66]

控制流翻译的关键是把跳转指令和目标地址匹配起来。问题是: 前向跳转的目标标签还没生成。

解决方法:

方法	思路	代价
两遍法	第一遍生成伪标签并记录标签地址; 第二遍把伪标签替换为实际地址	需要扫描代码两次
单遍法	遇到前向跳转先留下空洞, 维护待回 填列表; 目标地址确定后立即填洞	更适合和 LR parser 同步生成

Backpatching

回填是在单遍代码生成中处理前向跳转的方法: 先生成带空目标的跳转指令, 并把这些指令编号放入列表; 等目标地址已知时, 批量把列表中的空目标填成该地址。

10.3 回填属性和辅助函数 [p67]

综合属性:

- $B.truelist$: 当 B 为真时应跳往目标的待回填指令列表。
- $B.falselist$: 当 B 为假时应跳往目标的待回填指令列表。
- $S.nextlist$: 语句执行完后应跳到 S 后继位置的待回填指令列表。

辅助函数:

- $makelist(i)$: 创建只含指令编号 i 的列表。
- $merge(p1, p2)$: 合并两个列表。
- $backpatch(p, i)$: 把列表 p 中所有跳转空洞填成目标指令 i 。

10.4 控制流回填规则 [p68-70]

为了记录关键指令位置, 需要修改文法:

$$M \rightarrow \varepsilon, \quad M.inst = nextinst.$$

M 是 marker, 记录当前下一条将要生成的指令编号。

还需要:

$$N \rightarrow \varepsilon, \quad N.nextlist = makelist(nextinst); \quad gen(goto _).$$

N 用在 if-else 中, 表示 then 分支结束后要跳过 else 分支, 但当时还不知道 $S.next$ 的位置, 所以先生成空跳转。

规则摘要:

- if 无 else: 把真出口回填到 then 语句开始处; 语句后继列表由假出口和 then 语句自身未决后继合并得到。
- if-else: 把真出口回填到 then 开始, 把假出口回填到 else 开始; then 末尾跳过 else 的空跳转也并入后继列表。
- while: M_1 是循环测试开始, M_2 是循环体开始。B 为真进入循环体, 循环体结束跳回 $M_1.inst$, B 为假进入 $S.nextlist$ 。

通俗地说, 回填就是“先欠着目标地址, 等目标地址出现后一次性补上”。这让编译器可以单遍生成控制流代码。

11 总结与阅读 [p71-73]

课件总结的重点:

- 三地址码通用形式为 $X = Y \text{ op } Z$ 。
- 三地址码实现方式: 四元式、三元式、间接三元式。

- SSA 让每个变量静态上只赋值一次，便于优化。
- 使用语法制导翻译生成 TAC。
- 需要处理的代码结构包括变量定义、赋值、数组引用、布尔表达式和控制流。
- 单遍控制流代码生成依赖回填。

推荐阅读 Dragon Book 第 6 章，尤其是中间表示、类型检查以及各种程序结构的翻译。

12 练习题与解答

题 1：为什么现代编译器常使用多层 IR？

解答：不同阶段适合不同抽象层次。AST 适合语义分析，因为它保留源程序结构；TAC/SSA 适合机器无关优化，因为每条指令简单、数据依赖清楚；机器层 IR 适合寄存器分配和指令选择，因为它接近具体 ISA。多层 IR 还能复用优化器和后端：新增源语言只需转到统一低层 IR，新增机器只需从统一低层 IR 生成目标机器 IR。

题 2：把表达式 $x + y * z - w$ 翻译为三地址码。

解答：

```
t1 = y * z
t2 = x + t1
t3 = t2 - w
```

每条指令右侧最多一个操作符。

题 3：写出 $a = b * (-c) + b * (-c)$ 的四元式。

解答：

编号	op	arg1	arg2	result
0	minus	c		t1
1	*	b	t1	t2
2	minus	c		t3
3	*	b	t3	t4
4	+	t2	t4	t5
5	=	t5		a

题 4：三元式为什么不利于优化中的指令移动？间接三元式如何解决？

解答：三元式用指令编号隐含表示结果，其他指令通过编号引用该结果。优化时若移动、删除或重排指令，编号含义可能改变，所有引用都要更新。间接三元式把三元式存入数据库，真正代码顺序是指针列表；移动代码时只移动指针，不移动三元式本体，因此引用关系保持稳定。

题 5：SSA 中 ϕ 函数的作用是什么？

解答： ϕ 函数用于控制流合并点，表示根据实际来自哪条路径选择对应变量版本。例如：

$$x_5 = \phi(x_3, x_4)$$

表示若控制流来自 true 分支取 x_3 ，来自 false 分支取 x_4 。它让合并后的值流在 IR 中显式可见。

题 6：对如下变量声明计算未对齐和对齐布局。

```
char a;
int b;
int c;
long long d;
```

解答：若不考虑对齐，偏移为 $a = 0, b = 1, c = 5, d = 9$ 。若按类型大小对齐， $a = 0$ ，随后填充到 4， $b = 4, c = 8$ ，随后 long long 需要 8 字节对齐， $d = 16$ 。

题 7：翻译 $a = b + (-c)$ 。

解答：

```
t1 = minus c
t2 = b + t1
a = t2
```

其中 $t1, t2$ 是 newtemp() 生成的临时变量。

题 8：数组 `int a[3][5][8]` 中 `a[i][j][k]` 的偏移如何计算？

解答：假设 int 宽度为 4，C 语言 row-major 布局：

$$offset = i * (5 * 8 * 4) + j * (8 * 4) + k * 4 = i * 160 + j * 32 + k * 4.$$

TAC:

```
t1 = i * 160
t2 = j * 32
```

```
t3 = t1 + t2
t4 = k * 4
t5 = t3 + t4
```

若要读值：

```
t6 = a[t5]
```

题 9：用跳转方式翻译布尔表达式。

解答：

```
if a < b goto E.true
goto L1
L1: if c < d goto L2
goto E.false
L2: if e < f goto E.true
goto E.false
```

这体现短路：若 $a < b$ 为真，直接到真出口；若 $c < d$ 为假，直接到假出口。

题 10：为什么控制流翻译需要回填？

解答：因为单遍生成时，前向跳转目标还没生成。例如 if 语句中，条件为假时要跳到 then 之后或 else 开始，但生成条件跳转时目标地址尚未知。回填先生成带空洞的跳转指令，把这些指令编号放入列表；等目标指令编号确定后，用 backpatch 一次性填入目标地址。

题 11：说明 B.truelist、B.falselist 和 S.nextlist 的含义。

解答：

- B.truelist：B 为真时应跳往目标的未填地址跳转指令列表。
- B.falselist：B 为假时应跳往目标的未填地址跳转指令列表。
- S.nextlist：语句 S 执行完后应跳到 S 后继位置的未填地址跳转指令列表。

题 12：给出 while 语句的回填思路。

解答：对 $while(B)S_1$ ，需要记录循环测试开始位置 $M_1.inst$ 和循环体开始位置 $M_2.inst$ 。把 $B.truelist$ 回填到 $M_2.inst$ ，让条件为真进入循环体；把 $S_1.nextlist$ 回填到 $M_1.inst$ ，让循环体结束后回到测试；最后生成 $goto M_1.inst$ ，而 $S.nextlist = B.falselist$ ，表示条件为假时跳出循环。

复习建议

复习本讲时建议按三条线整理：

- 表示线：AST、DAG、TAC、四元式、三元式、间接三元式、SSA 分别解决什么问题。
- 翻译线：变量布局、赋值、数组引用、布尔表达式、控制流分别如何生成 TAC。
- 控制流线：标签、短路求值、前向跳转、truelist/falselist/nextlist、backpatch 的关系。

最重要的动手能力是：把复杂表达式拆成 TAC；计算多维数组偏移；把布尔表达式翻译成跳转；解释回填列表如何在 if/while 中被合并和填充。